



Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Registro de práctica #5 Defender y Evolucionar una Decisión Arquitectónica mediante ADR.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Mendez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 03 de Octubre de 2026.

Objetivo o finalidad.

Entender que el uso del ADR es una forma de realizar y tomar una decisión en términos de Requisitos Funcionales, debido a que manejan en distintos puntos dado que se quedan o solo se implementan sus alternativas en caso de ser requerido.

Por ello en ningún momento se elimina ni se borra por el aspecto, ya que forma parte del antecedente que se realizó, y los posteriores que se realicen serán en gran ayuda.

Al igual a entender que de acuerdo con las implementaciones de nuevos requisitos funcionales o requisitos no funcionales, cambiará el panorama de forma momentánea y se tomaran medidas para hacerlo de forma eficiente apenas se presente la situación.

1.- Descripción del problema.

Consiste basándose mediante el sistema de reservación de laboratorios que anteriormente se analizó, haciendo el uso del CRUD, que lo maneja a través de consultar, cancelar, crear y aprobar la disponibilidad de solicitar reservaciones de los laboratorios disponibles, así mismo de que los administradores aprueben y gestionen las solicitudes.

Además de que su arquitectura que maneja de carácter inicial de cliente – servidor que se utiliza es de REST – JSON.

2.- Requisitos Funcionales.

RF	Funcionalidad
RF01	Consultar laboratorios.
RF02	Consultar disponibilidad.
RF03	Crear reservación.
RF04	Cancelar reservación.
RF05	Aprobar reservación.
RF06	Consultar historial.

3.- Requisitos No Funcionales.

RNF	Funcionalidad
RF01	Soporte para clientes web y móviles.
RF02	Api sencillo de consumir.
RF03	Facilidad de mantenimiento.
RF04	Tiempo de respuesta CRUD < 3 segundos.
RF05	Experiencia básica del equipo en HTTP y REST.
RF06	Sin requerimiento inicial de tiempo real.

4.- Pregunta Arquitectónica.

¿Qué mecanismo de comunicación utilizaremos inicialmente entre las aplicaciones cliente y el backend para las operaciones principales del sistema de reservaciones?

5 y 6.- Matriz inicial con sus alternativas respectivamente.

En las alternativas se buscaron tecnologías y así mismo con la matriz con la que se inició de forma inicial.

Tecnologías alternativas.	Puntuación.
REST	4.70
gRPC	3.35
WebSocket	3.25
SSE	3.45

Mediante la comparación de REST con las otras alternativas, este se posiciona la mejor opción para el manejo de CRUD, además de su compatibilidad y mantenibilidad.

7.- ADR-001

ADR-001 – REST COMO MECANISMO PRINCIPAL.

Estado: Aceptado.

El contexto con el que se maneja es porque el sistema será utilizado desde clientes web y móviles; además las operaciones iniciales corresponden principalmente a consulta, creación, actualización y cancelación, pero en este caso no se requiere comunicación bidireccional permanente ni actualización automática en tiempo real.

Alternativas consideradas:

Estas son REST, gRPC, WebSocket y Server-Sent Events (SSE).

Decisión.

Utilizar REST sobre HTTP/HTTPS con JSON como mecanismo principal de comunicación entre las aplicaciones de cliente y la API.

Justificación y consecuencias.

El uso de REST, tiene una buena funcionalidad con el uso de CRUD, además de su compatibilidad web o Flutter, también por su parte de la experiencia del equipo y la facilidad de realizar pruebas, en conjunto con su ecosistema y mantenimiento.

Por otra parte, también están las consecuencias positivas en donde estas consisten mediante la implementación sencilla, interoperabilidad, documentación OpenAPI, depuración y menor curva de aprendizaje.

También está presente las consecuencias negativas, de la cual no existe comunicación bidireccional continua, además de actualizaciones frecuentes que podrían requerir polling.

En parte a los riesgos, que puede ser de aparición futura de tiempo real, además del exceso de polling o streaming.

Ya en la evidencia, se tiene el uso mediante la matriz, donde REST = 4.70, PoC POST/api/reservaciones; pruebas con navegador/curl/cliente API; modelo C4.

8 y 9.- Presentación de la defensa.

¿Por qué REST?

Por qué se ajusta perfectamente a las operaciones CRUD que requiere el sistema de forma inicial, además de ser un estándar compatible para cualquier cliente ya sea de móvil o web en Flutter y fácil de implementarlo mediante OpenAI y también de mantenerlo en el equipo.

¿Por qué no WebSocket?

Por qué este está diseñado para comunicación de carácter bidireccional continua en tiempo real.

¿Por qué no gRPC?

Por qué de por sí ya es excelente en rendimiento por parte de HTTP, su integración de forma directa hace algo complejo con clientes web, y por ende causaría una complejidad técnica en comparación con el uso de REST, que ya es algo sencillo de manejar desde el equipo y cualquier cliente.

¿Por qué JSON?

Este es un formato de intercambio de datos de uso sencillo y muy táctico para cualquier lenguaje de programación y cliente HTTP, además de tener soporte inmediato.

¿Qué desventaja tiene REST?

Lo malo es su forma de modelo de petición, por que además de que no se actualiza de forma automática, siempre se tiene que iniciar mediante el cliente para que el servidor pueda enviar una petición y dar una respuesta.

10.- Cambia el escenario debido a la integración de RNF07.

RNF07 = Cuando una reservación sea creada, aprobada, cancelada o modificada, los usuarios podrán observar el cambio automáticamente en menos de 2 segundos y sin actualizar la página.

11.- Análisis de Impacto.

El RNF07 cambia el panorama por completo, por que antes se manejaba el cambio a partir de que el cliente solicite la petición al servidor y este mandaba la respuesta, ahora quiere que se actualice de carácter primordialmente primero desde el servidor.

Por sí solo REST sigue siendo buena opción, pero debido a esa pequeña integración de requisito, sí o sí requiere que también trabaje con algo más aparte, por que por si solo se vuelve complejo.

12.- Nueva actualización.

Tecnología	Puntuación nueva
REST	3.70
WebSocket	4.00
SSE	4.15

Con la integración del RNF07, el SSE obtiene una mayor puntuación entre las alternativas, pero esto no obliga a eliminar a REST, pero a su compatibilidad directa y fortaleza al CRUD permanece.

Por ello de ahora en adelante se utilizará tanto REST como SSE.

13.- ADR-002.

ADR-002 Incorporar Server-Sent Events para actualizaciones de disponibilidad.

Estado: Aceptado.

El motivo es por que el RNF07 exige cambios visibles de forma automática cada 2 segundos y sin recargar.

Alternativas a considerarse.

REST + polling, sustituir por WebSocket; REST + WebSocket; REST + SSE.

Decisión.

Mantener REST para CRUD e incorporar SSE para servidor – cliente.

Consecuencias positivas.

Se da mediante actualización automática; además de conservar REST, y reducir el polling.

Consecuencias negativas.

Mantiene conexiones abiertas, usa dos mecanismos, y maneja mayor complejidad y necesidad de considerar escalamiento.

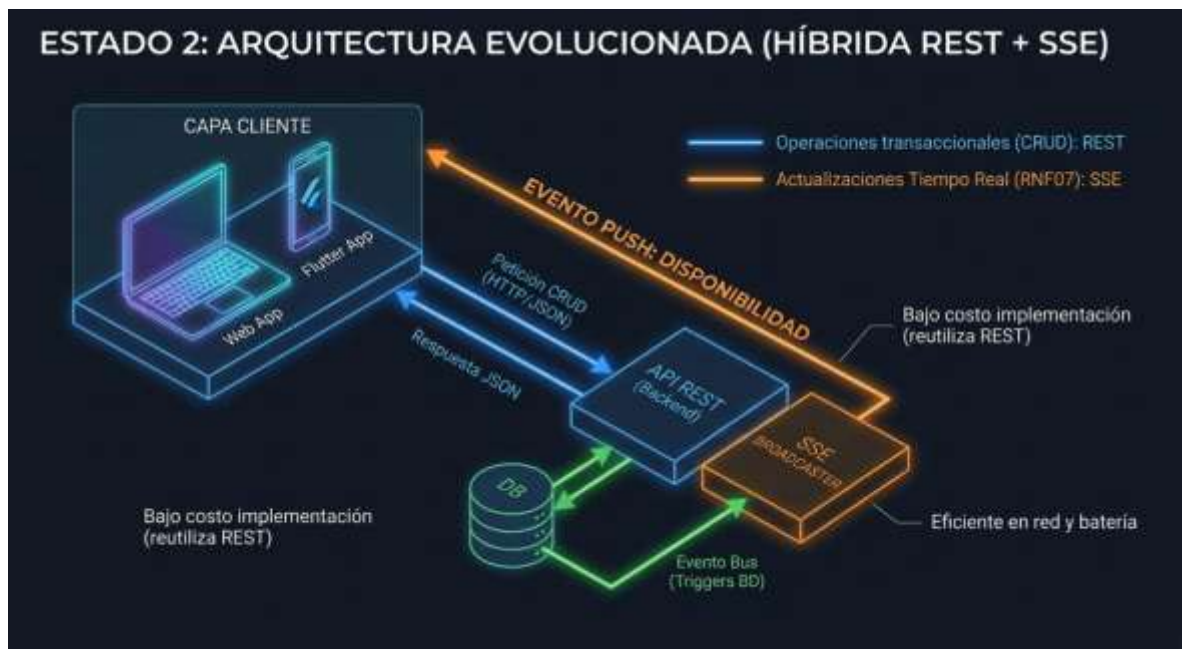
14.- Estado Actualizado del ADR Anterior.

Teniendo en cuenta todo, REST siendo la arquitectura con la que se inició el proyecto se mantiene a pesar de todo por que la utilidad del CRUD funciona adecuadamente, por otra parte, también se añade e implementa el SSE para las funciones de actualizaciones automáticas.

15.- Diagrama de Arquitectura de Antes.



16.- Diagrama de Arquitectura Después.



17.- Conclusión.

En que el uso de REST en este tipo de proyectos como de reservación de laboratorios, donde se maneja el uso de CRUD, es muy crucial y de forma obligatoria ya que qué el intercambio de datos es importante aquí, además de buscar alternativas al principio y se ajusta de forma acción hace una reacción, por ello al implementar otro RNF como de que se actualice sin recargar la pagina cambia tanto su dinámica como parte de su arquitectura, por eso se buscó dentro de las alternativas cuales podría cumplir con esa necesidad y se añade sin eliminar a la principal.

Ya estando adjuntas trabajan de forma dependientes, para cumplir con lo que se requiere y en caso de que surja alguna más posteriormente, se analiza y se toma nuevamente una decisión para tratar de que todas las decisiones sean de forma lógicas y claras.

Preguntas de reflexión.

1.- ¿Por qué no existe una arquitectura perfecta?

Porque en software todo implica renunciar a algo, si priorizas máxima velocidad o tiempo real, sueles aumentar la complejidad o el costo de la infraestructura, dejas algo para tener todo lo demás importante en óptimas condiciones.

2.- ¿Qué diferencia existe entre una decisión y una preferencia?

Una decisión se toma con base en necesidades técnicas objetivas y datos concretos del proyecto, en contra parte con una preferencia es simplemente el gusto personal de un desarrollador por usar cierta herramienta o lenguaje.

3.- ¿Qué función cumplen los RNF?

Los Requisitos No Funcionales (RNF) definen cómo debe comportarse el sistema en aspectos de calidad, como la velocidad de respuesta, la seguridad, la estabilidad o la capacidad de soportar usuarios al mismo tiempo.

4.- ¿Por qué una decisión correcta puede necesitar cambiar?

Porque los proyectos cambian, cuando hay una decisión puede ser excelente al inicio cuando los requisitos son sencillos, pero si más adelante el negocio pide nuevas funciones o mayor velocidad, esa decisión original puede quedarse corta.

5.- ¿Por qué no debemos borrar ADR antiguos?

Porque sirven como la bitácora del proyecto, mantenerlos permite entender por qué se tomaron ciertas decisiones en el pasado y evita volver a discutir o cometer errores que ya se habían superado.

6.- ¿Qué significa que una arquitectura evolucione?

Significa que la estructura del software se adapta y crece de forma ordenada a medida que cambian las necesidades, en lugar de tener que tirar todo a la basura y empezar desde cero.

7.- ¿Qué consecuencias negativas tuvo la primera decisión?

En su momento ninguna, pero al aparecer la necesidad de actualizar datos en tiempo real, la primera versión no permitía enviar notificaciones al usuario al instante sin obligarlo a recargar la pantalla.

8.- ¿Qué requisito provocó el cambio?

La necesidad de mostrar los cambios de disponibilidad de forma automática e inmediata en la pantalla (en menos de 2 segundos) sin que el usuario tenga que refrescar.

9.- ¿Qué habría ocurrido si no modificaban la arquitectura?

El sistema se habría vuelto muy lento e ineficiente, saturando el servidor con peticiones constantes solo para revisar si había cambios, lo que además habría agotado rápido la batería y datos de los celulares.

10.- ¿La nueva solución es más compleja?

Sí, porque ahora el servidor no solo responde a solicitudes comunes, sino que mantiene un canal continuo para enviar eventos en tiempo real hacia los dispositivos.

11.- ¿La complejidad adicional está justificada?

Sí, totalmente es la forma más eficiente y ligera de lograr la inmediatez que pide el proyecto sin sobrecargar el sistema.

12.- ¿Qué otro requisito podría volver a cambiar la arquitectura?

Si en el futuro se necesitara una comunicación bidireccional continua (por ejemplo, un chat en vivo o una herramienta donde varios usuarios editen lo mismo al mismo tiempo), se tendría que migrar a tecnologías como WebSockets.